

The Amazon Diwali India Sales Dataset (2025) is a synthetic dataset containing 15,000 e-commerce transactions designed to simulate real-world sales on Amazon India. It captures customer purchasing behavior, product details, payment preferences, and feedback patterns across various product categories throughout the year 2025. The goal and objective of this analysis are to explore the data and try to find patterns, relationships between variables, and key words in the reviews that show what clients are saying about the products. Stakeholders should be informed about problems with products or any information that leads to dissatisfied clients.

In [1]: *# Standard operational package imports.*

```
import numpy as np
import pandas as pd

# Important imports for preprocessing, modeling, and evaluation.
from sklearn.preprocessing import OneHotEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
import sklearn.metrics as metrics
from collections import Counter
import re

# Visualization package imports.
import matplotlib.pyplot as plt
import seaborn as sns
```

In [2]: *# Load dataset into dataframe*

```
df = pd.read_csv(r"C:\Users\USER\OneDrive\Documentos\Trabalho\Ciencia_de_dados\Proj
```

In [2]: *# Show the first 5 entries of the dataset*

```
df.head()
```

Out[2]:

	Order_ID	Date	Customer_ID	Product_Category	Product_Name	Quantity	Unit_Price_INR	T
--	----------	------	-------------	------------------	--------------	----------	----------------	---

0	ORD100000	2025-01-25	CUST2796	Home & Kitchen	Cookware Set	2	25574.41	
1	ORD100001	2025-08-28	CUST9669	Beauty	Hair Dryer	1	19361.41	
2	ORD100002	2025-02-27	CUST5808	Electronics	Tablet	3	38476.22	
3	ORD100003	2025-02-24	CUST5889	Electronics	Headphones	5	38145.72	
4	ORD100004	2025-06-15	CUST9005	Clothing	Saree	5	45940.98	

In [3]: *# Get information about the dataset*

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15000 entries, 0 to 14999
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Order_ID              15000 non-null  object
1   Date                  15000 non-null  object
2   Customer_ID           15000 non-null  object
3   Product_Category      15000 non-null  object
4   Product_Name          15000 non-null  object
5   Quantity              15000 non-null  int64
6   Unit_Price_INR        15000 non-null  float64
7   Total_Sales_INR       15000 non-null  float64
8   Payment_Method        15000 non-null  object
9   Delivery_Status       15000 non-null  object
10  Review_Rating         15000 non-null  int64
11  Review_Text           15000 non-null  object
12  State                 15000 non-null  object
13  Country               15000 non-null  object
dtypes: float64(2), int64(2), object(10)
memory usage: 1.6+ MB
```

In [5]: *# Check for missing or null values*

```
print(df.isnull().sum())
print()
print(df.isna().sum())
```

```
Order_ID      0
Date          0
Customer_ID   0
Product_Category  0
Product_Name  0
Quantity      0
Unit_Price_INR  0
Total_Sales_INR  0
Payment_Method  0
Delivery_Status  0
Review_Rating  0
Review_Text   0
State         0
Country       0
dtype: int64
```

```
Order_ID      0
Date          0
Customer_ID   0
Product_Category  0
Product_Name  0
Quantity      0
Unit_Price_INR  0
Total_Sales_INR  0
Payment_Method  0
Delivery_Status  0
Review_Rating  0
Review_Text   0
State         0
Country       0
dtype: int64
```

In [6]: *# Check for duplicate values*

```
print(df.duplicated().sum())
print()
print(df[df.duplicated()])
```

0

Empty DataFrame

Columns: [Order_ID, Date, Customer_ID, Product_Category, Product_Name, Quantity, Unit_Price_INR, Total_Sales_INR, Payment_Method, Delivery_Status, Review_Rating, Review_Text, State, Country]

Index: []

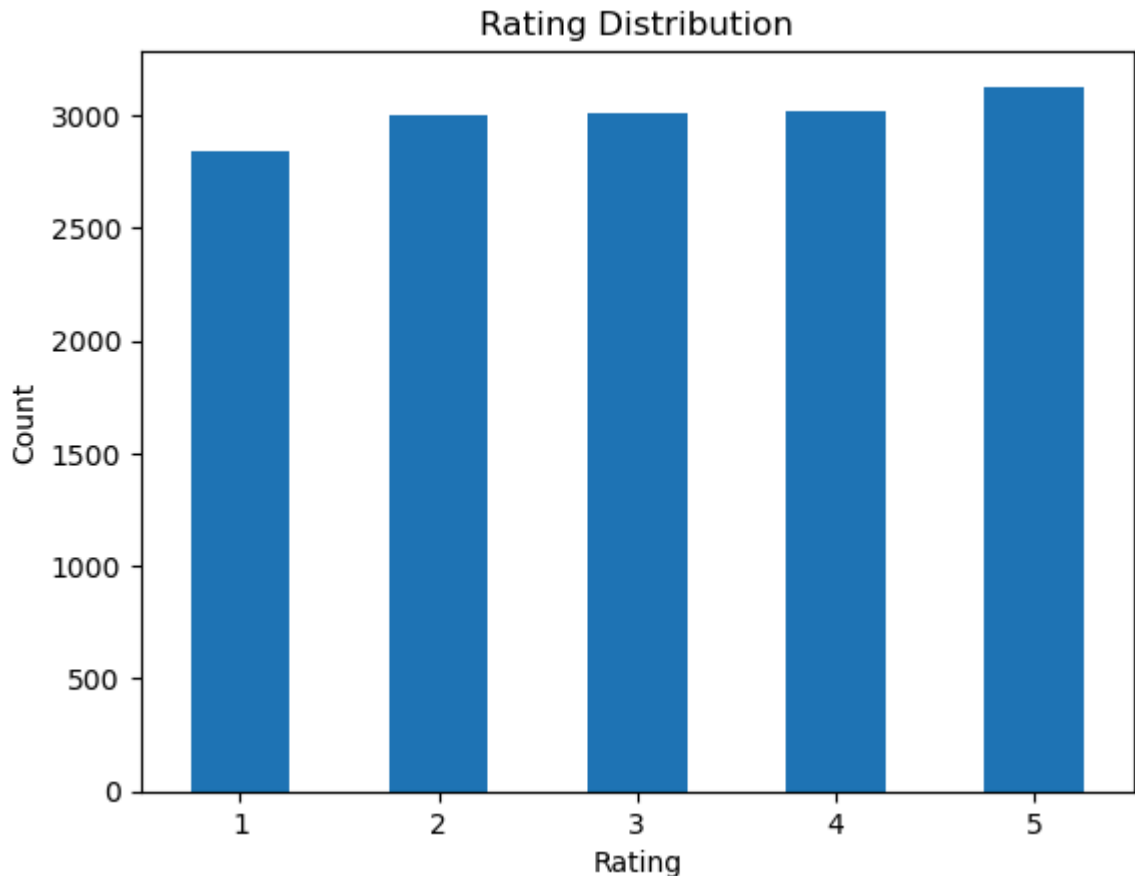
This dataset was designed for studying purposes, so it has no missing or duplicated values.

The work will focus in finding the relationship between variables to try to understand why people gave bad reviews on some products, services. So, every note that is 1 or 2 will be put together in a group to make a binary variable with the rest of the clients.

```
In [2]: # Percentage of each rating
print(df['Review_Rating'].value_counts(normalize=True) * 100)
```

```
5    20.853333
4    20.146667
3    20.086667
2    19.986667
1    18.926667
Name: Review_Rating, dtype: float64
```

```
In [8]: # Bar graph of the ratings
df['Review_Rating'].value_counts().sort_index().plot(kind='bar')
plt.title('Rating Distribution')
plt.xlabel('Rating')
plt.ylabel('Count')
plt.xticks(rotation=0)
plt.show()
```



```
In [3]: # Ratings 1 and 2 will be the target, so they will get 1. Ratings 3, 4 and 5 will be
```

```
mapp = {1: 1, 2: 1, 3: 0, 4: 0, 5: 0}
df['group_rating'] = df['Review_Rating'].map(mapp)
```

```
In [5]: df.head()
```

```
Out[5]:
```

	Order_ID	Date	Customer_ID	Product_Category	Product_Name	Quantity	Unit_Price_INR	T
0	ORD100000	2025-01-25	CUST2796	Home & Kitchen	Cookware Set	2	25574.41	
1	ORD100001	2025-08-28	CUST9669	Beauty	Hair Dryer	1	19361.41	
2	ORD100002	2025-02-27	CUST5808	Electronics	Tablet	3	38476.22	
3	ORD100003	2025-02-24	CUST5889	Electronics	Headphones	5	38145.72	
4	ORD100004	2025-06-15	CUST9005	Clothing	Saree	5	45940.98	

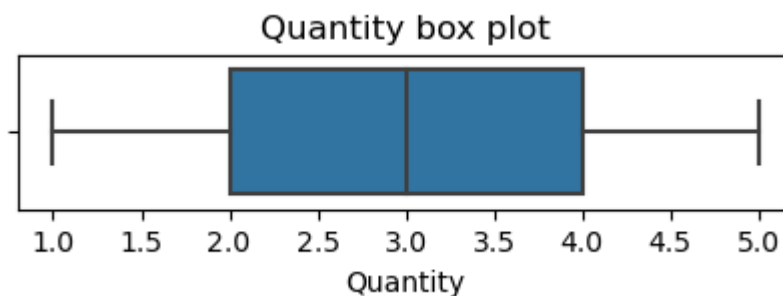
```
In [6]: # Percentage of Delivery Status
print(df['Delivery_Status'].value_counts(normalize=True) * 100)
```

```
Delivered    33.833333
Pending      33.626667
Returned     32.540000
Name: Delivery_Status, dtype: float64
```

Examining the spread and distribution of important variables using box plots and histograms.

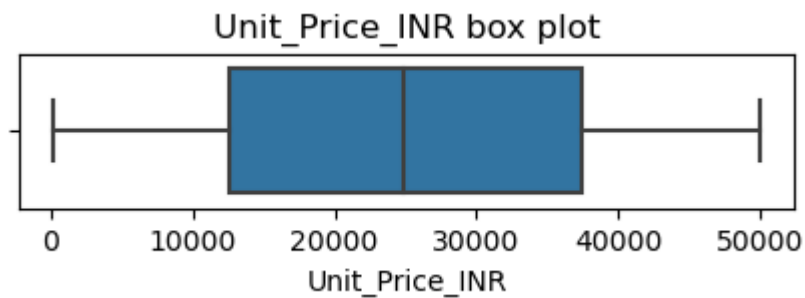
```
In [11]: #Box plot of quantity

plt.figure(figsize=(5,1))
sns.boxplot(x=df['Quantity'], fliersize=1)
plt.title('Quantity box plot');
```



```
In [16]: # Box plot of Unit_Price_INR

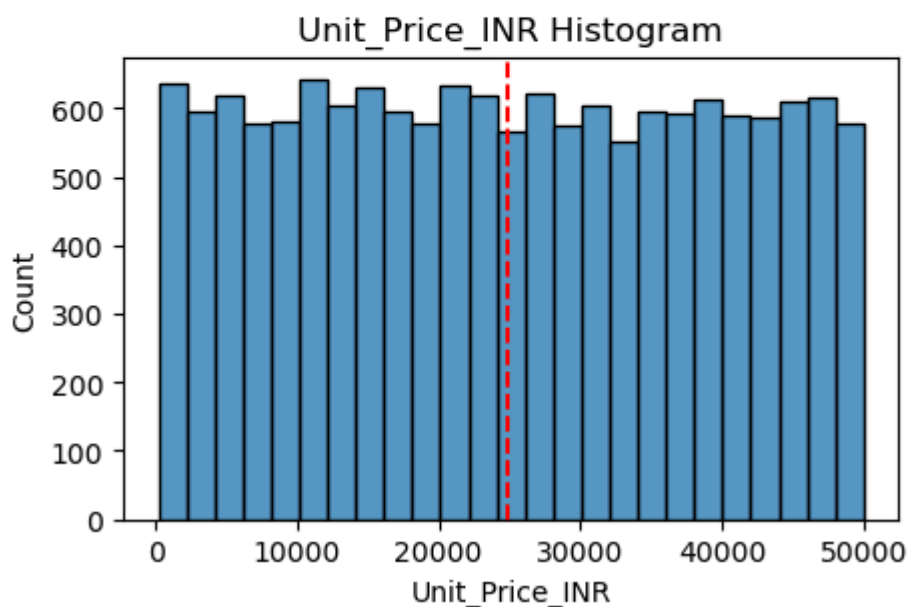
plt.figure(figsize=(5,1))
sns.boxplot(x=df['Unit_Price_INR'], fliersize=1)
plt.title('Unit_Price_INR box plot');
```



```
In [21]: # Histogram of Unit_Price_INR

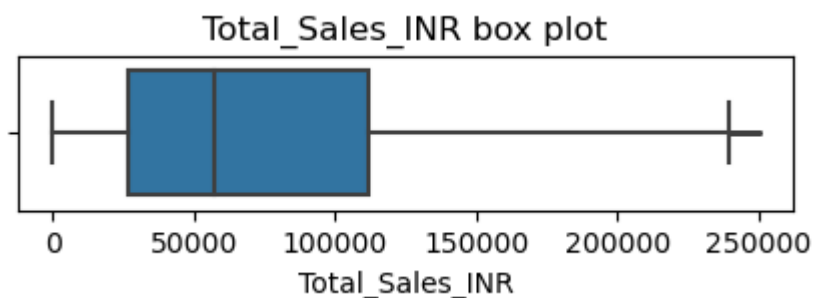
plt.figure(figsize=(5, 3), dpi=100)
sns.histplot(x=df['Unit_Price_INR'])
median = df['Unit_Price_INR'].median()
plt.axvline(median, color='red', linestyle='--')
plt.text(75, 1200, 'median', color='red')
plt.title('Unit_Price_INR Histogram')
plt.show()
```

median



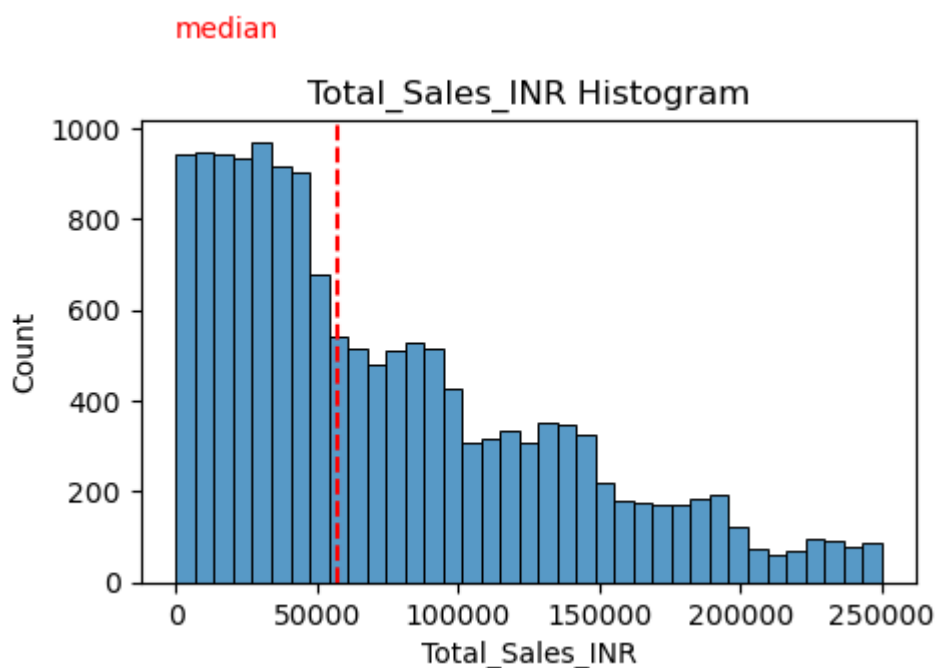
```
In [22]: # Box plot of Total_Sales_INR

plt.figure(figsize=(5,1))
sns.boxplot(x=df['Total_Sales_INR'], fliersize=1)
plt.title('Total_Sales_INR box plot');
```



```
In [23]: # Histogram of Total_Sales_INR

plt.figure(figsize=(5, 3), dpi=100)
sns.histplot(x=df['Total_Sales_INR'])
median = df['Total_Sales_INR'].median()
plt.axvline(median, color='red', linestyle='--')
plt.text(75, 1200, 'median', color='red')
plt.title('Total_Sales_INR Histogram')
plt.show()
```



The box plots and histograms of the numeric variables show that the price of the products is well distributed between 0 to 50,000 Rupees. Clients usually purchase less than 50,000 Rupees, but there are a good amount of bigger sales as well.

```
In [7]: # Understand what are the main products with bad or good reviews, selecting the gro

# Group 1
group1 = df[df['group_rating'] == 1]
top5_g1 = pd.DataFrame({
    'count': group1['Product_Name'].value_counts().head(5),
    'percentage': (group1['Product_Name'].value_counts().head(5) / len(group1) * 100
})
print("=== TOP 5 PRODUCTS - GROUP 1 (Ratings 1-2) ===")
print(top5_g1)

# Group 0
group0 = df[df['group_rating'] == 0]
top5_g0 = pd.DataFrame({
    'count': group0['Product_Name'].value_counts().head(5),
    'percentage': (group0['Product_Name'].value_counts().head(5) / len(group0) * 100
```

```
})
print("\n=== TOP 5 PRODUCTS - GROUP 0 (Ratings 3-5) ===")
print(top5_g0)
```

```
=== TOP 5 PRODUCTS - GROUP 1 (Ratings 1-2) ===
```

	count	percentage
Hair Dryer	260	4.45
Saree	254	4.35
Jeans	251	4.30
Perfume	250	4.28
Science Textbook	245	4.20

```
=== TOP 5 PRODUCTS - GROUP 0 (Ratings 3-5) ===
```

	count	percentage
Children's Book	393	4.29
Laptop	391	4.27
Tablet	389	4.25
Face Cream	382	4.17
T-Shirt	381	4.16

The table above show that there is not a single product responsible for most of the bad or good reviews. The percentages are well distributed, so it is not possible to accuse a few products for the ratings. Although, it is important to check on the quality of the top 5 products that received bad reviews.

In [5]: *# Understand what are the main Product Category with bad or good reviews, selecting*

```
# Group 1
group1 = df[df['group_rating'] == 1]
top5_g1 = pd.DataFrame({
    'count': group1['Product_Category'].value_counts().head(5),
    'percentage': (group1['Product_Category'].value_counts().head(5) / len(group1)
})
print("=== TOP 5 PRODUCTS CATEGORY - GROUP 1 (Ratings 1-2) ===")
print(top5_g1)

# Group 0
group0 = df[df['group_rating'] == 0]
top5_g0 = pd.DataFrame({
    'count': group0['Product_Category'].value_counts().head(5),
    'percentage': (group0['Product_Category'].value_counts().head(5) / len(group0)
})
print("\n=== TOP 5 PRODUCTS CATEGORY - GROUP 0 (Ratings 3-5) ===")
print(top5_g0)
```

```
=== TOP 5 PRODUCTS CATEGORY - GROUP 1 (Ratings 1-2) ===
```

	count	percentage
Clothing	1221	20.92
Books	1177	20.16
Beauty	1172	20.08
Electronics	1166	19.98
Home & Kitchen	1101	18.86

```
=== TOP 5 PRODUCTS CATEGORY - GROUP 0 (Ratings 3-5) ===
```

	count	percentage
Electronics	1870	20.41
Books	1858	20.28
Beauty	1825	19.92
Home & Kitchen	1809	19.74
Clothing	1801	19.66

The table above show that there is not a single product category responsible for most of the bad or good reviews. The percentages are well distributed, so it is not possible to accuse a

few product category for the ratings. Although, it is important to check on the quality of the top 5 product category that received bad reviews.

The Review_Text column shows what clients are saying about the products. The following code identifies the most frequently appearing words in the reviews to find anything interesting to report to stakeholders.

```
In [4]: # Stopwords
stopwords = {'the', 'is', 'in', 'and', 'to', 'a', 'of', 'for', 'it', 'this',
             'that', 'with', 'on', 'was', 'are', 'as', 'but', 'at', 'by', 'an',
             'be', 'have', 'has', 'had', 'not', 'or', 'from', 'i', 'my'}

def get_top_words(texts, top_n=20, remove_stopwords=True):
    all_text = ' '.join(texts.astype(str)).lower()
    words = re.findall(r'\b[a-z]+\b', all_text)

    if remove_stopwords:
        words = [w for w in words if w not in stopwords and len(w) > 2]

    return Counter(words).most_common(top_n)

# Group 0
print("=== TOP WORDS - GROUP 0 (Satisfied) ===")
satisfied = df[df['group_rating'] == 0]['Review_Text']
for word, count in get_top_words(satisfied):
    print(f"{word}: {count}")

print("\n=== TOP WORDS - GROUP 1 (Unsatisfied) ===")
# Group 1
unsatisfied = df[df['group_rating'] == 1]['Review_Text']
for word, count in get_top_words(unsatisfied):
    print(f"{word}: {count}")
```


=== TOP WORDS - GROUP 0 (Satisfied) ===

product: 1904
quality: 1254
satisfied: 658
excellent: 653
fantastic: 641
highly: 637
recommend: 637
could: 632
better: 632
worth: 615
every: 615
rupee: 615
good: 613
average: 611
pretty: 609
decent: 609
bad: 597
would: 594
buy: 594
again: 594

=== TOP WORDS - GROUP 1 (Unsatisfied) ===

product: 1178
poor: 626
packaging: 626
quality: 607
could: 607
improve: 607
terrible: 601
experience: 601
satisfied: 600
expected: 592
very: 586
bad: 586
waste: 573
money: 573
disappointed: 573
defective: 543
item: 543
never: 536
buying: 536
again: 536

The key finding from the word analysis is that 'packaging' appears frequently among dissatisfied customers. This suggests the need to examine the quality of product packaging.

```
In [5]: # Check the data type of each column  
  
df.dtypes
```

```
Out[5]: Order_ID      object
        Date        object
        Customer_ID  object
        Product_Category  object
        Product_Name  object
        Quantity      int64
        Unit_Price_INR  float64
        Total_Sales_INR  float64
        Payment_Method  object
        Delivery_Status  object
        Review_Rating  int64
        Review_Text    object
        State          object
        Country         object
        group_rating    int64
        dtype: object
```

To check the relationship between variables it will be perform a logistic regression model with an independent variable, X, Unit_Price_INR, and a dependent variable, y, group_rating. The unit price is a float data type, it can have a relation to the bad or good reviews and it is already in the right type for the model. The group_rating is our target, it is with the right data type for the model as well.

This dataset was prepared with balance, but for technical issues it is important to divide the data into training and testing set, so it is possible to check results and avoid bias. So, the next step will put 70% of the data into a training set and the remaining 30% into a testing set. The next code will create an X and y DataFrame with only the necessary variables.

```
In [6]: X = df[["Unit_Price_INR"]]
        y = df["group_rating"]
        X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.3, random_stat
```

```
In [7]: # Fit a Logistic Regression model to the data

        clf = LogisticRegression().fit(X_train,y_train)
```

```
In [8]: # Obtain parameter estimates

        clf.coef_
```

```
Out[8]: array([[ -1.46153452e-05]])
```

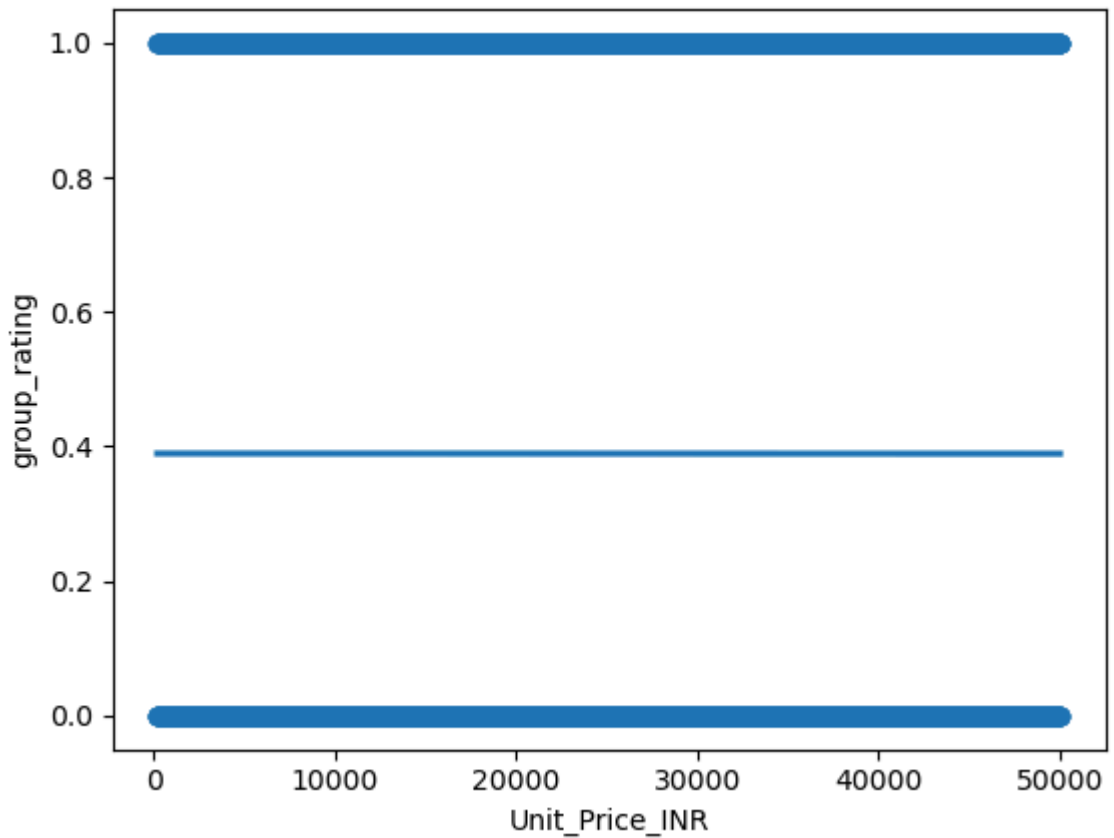
```
In [9]: clf.intercept_

Out[9]: array([ -1.07554848e-09])
```

```
In [11]: # Create a plot of your model

        sns.regplot(x="Unit_Price_INR", y="group_rating", data=df, logistic=True, ci=None)

Out[11]: <AxesSubplot:xlabel='Unit_Price_INR', ylabel='group_rating'>
```



Unit_Price_INR shows weak correlation with customer reviews. This dataset would benefit from additional customer and product details to identify stronger patterns and enable more effective machine learning models for predicting reviews and detecting product issues. Stakeholders should be aware that this dataset is limited, and the primary actionable insight is the need to evaluate packaging quality.

In []: